

Нейросетевое ранжирование

ШАД, Рекомендательные системы
2025/26, лекция 6

Кирилл Хрыльченко

Старший преподаватель,
ФКН ВШЭ

План занятия

Начинаем обсуждение нейросетевого ранжирования:

1. Почему нам недостаточно классического ML?
2. Категориальные признаки
3. Вещественные признаки и выходы других моделей

На семинаре разбор статей:

- [Unified Embeddings](#)
- [Piecewise Linear Encoding](#)

Ресар ранжирования

- На стадии ранжирования работаем с пофилтрованным каталогом
- Можем для каждого кандидата посчитать тяжелую функцию
 - Подаём на вход в модель максимум информации
 - Раннее связывание user, context, item признаков
- Важно сделать скоринг: получить для каждого кандидата оценку, по которой можно сделать сортировку

Почему нам недостаточно классического ML?

Часть 1

Tabular deep learning

- TabDL – нейросети для табличных данных
- Приближенный к реальности бенчмарк TabReD показывает, что простые MLP с ансамблированием лучше навороченных алгоритмов
 - Но иногда все равно побеждает бустинг
- TabPFN – foundation модели для табличных данных

[TabReD: Analyzing Pitfalls and Filling the Gaps in Tabular Deep Learning Benchmarks](#)

[TabPFN-v2: Accurate predictions on small data with a tabular foundation model](#)

ABSTRACT

Advances in machine learning research drive progress in real-world applications. To ensure this progress, it is important to understand the potential pitfalls on the way from a novel method's success on academic benchmarks to its practical deployment. In this work, we analyze existing tabular benchmarks and find two common characteristics of tabular data in typical industrial applications that are underrepresented in the datasets usually used for evaluation in the literature. First, in real-world deployment scenarios, distribution of data often changes over time. To account for this distribution drift, time-based train/test splits should be used in evaluation. However, popular tabular datasets often lack timestamp metadata to enable such evaluation. Second, a considerable portion of datasets in production settings stem from extensive data acquisition and feature engineering pipelines. This can have an impact on the absolute and relative number of predictive, uninformative, and correlated features compared to academic datasets. In this work, we aim to understand how recent research advances in tabular deep learning transfer to these underrepresented conditions. To this end, we introduce TabReD – a collection of eight industry-grade tabular datasets. We reassess a large number of tabular ML models and techniques on TabReD. We demonstrate that evaluation on time-based data splits leads to different methods ranking, compared to evaluation on random splits, which are common in current benchmarks. Furthermore, simple MLP-like architectures and GBDT show the best results on the TabReD datasets, while other methods are less effective in the new setting.

[TabReD: Analyzing Pitfalls and Filling the Gaps in Tabular Deep Learning Benchmarks](#)

Специфика рекомендаций

- **Масштаб:** в рексистемах гораздо больше и размеры датасетов, и количество признаков
- **Признаковое пространство:** очень важны категориальные признаки высокой кардинальности (sparse features)
 - Структура признакового пространства неявно диктуется наличием пользователя, контекста и айтема

Bitter lesson

Область	Раньше	Сейчас
Natural Language Processing	Лингвистические правила и фичи	LLMs (Decoder-Only Transformers)
Computer Vision	Ручные фичи (HOG, SIFT)	CNN, ViT (Visual Transformers)
Speech Processing	MFCC + Hidden Markov Models	Wav2vec, Whisper
Recommender Systems	Градиентный бустинг, ручные фичи	?

“The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin.”

Richard Sutton, The Bitter Lesson (2019)

Sparse features

- Нам нужно кодировать признаки высокой кардинальности (item, artist, brand, etc)
- Как GBDT кодирует категориальные признаки?

Sparse features

- Нам нужно кодировать признаки высокой кардинальности (item, artist, brand, etc)
- Как GBDT кодирует категориальные признаки? One hot или target encoding
- Для высококардинальных признаков one hot сделать не можем, а target encoding теряет семантику

В нейросетях – естественным образом кодируем категориальные признаки через эмбединги.

Неструктурированные данные

- Это тексты, изображения, аудио-сигнал, история пользователя
- Можно нагенерировать по ним признаки для бустинга, но это будет работать хуже DL
- Можно сделать нейросетевые модели для неструктурированных данных и уже из них извлечь признаки
 - E.g, скалярное произведение из двухбашенной user-item модели
 - А можно ли сделать representation learning?

Эмбединги

- Эмбединги на входе в градиентный бустинг работают плохо
- По отдельности координаты могут ничего не значить, семантика заложена именно в цельный эмбединг
- Каждый сплит в дереве анализирует только одну координату
 - Нужно очень глубокое дерево, чтобы учесть весь эмбединг
- А еще иногда хотим учить end-to-end нейросети для анализа неструктурированного контента и признаков

Feature engineering

GBDT плохо справляется с:

- Категориальными признаками высокой кардинальности
- Неструктурированными данными
- Эмбедингами

И вообще, feature engineering противоречит bitter lesson. Зачем вручную генерировать признаки, если есть нейросети?

Scaling hypothesis

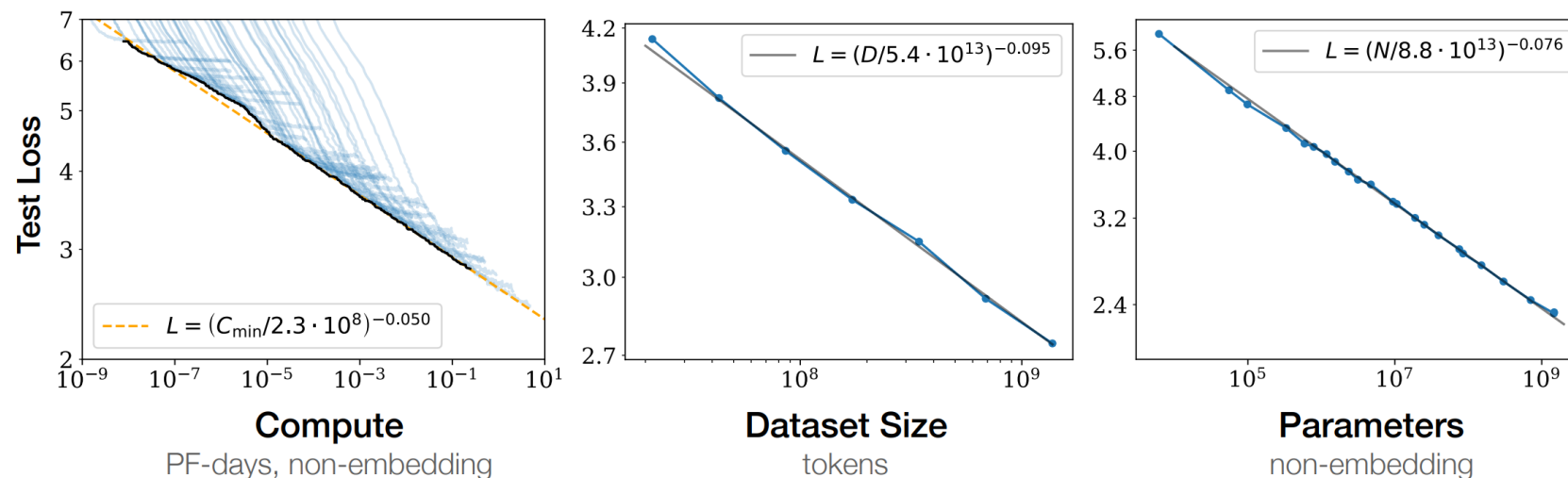


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

“If you have a large dataset and you train a very big neural network, then success is guaranteed!”

Ilya Sutskever, The Scaling Hypothesis (2014)

Многосигнальность

- В рекомендациях у нас обычно несколько важных метрик/сигналов
 - E.g., лайки и прослушивания
- Сложно моделировать их в единой модели
 - Синтетический таргет или взвешивание сэмплов – сложная настройка
 - А отдельные модели – это дорого и не всегда хорошо работает

А для нейросетей – можно сделать одну модель с несколькими головами.



[Как улучшить знакомые подходы для рекомендации незнакомого \(Савва Степурин, Яндекс Музыка\)](#)

Distribution drift

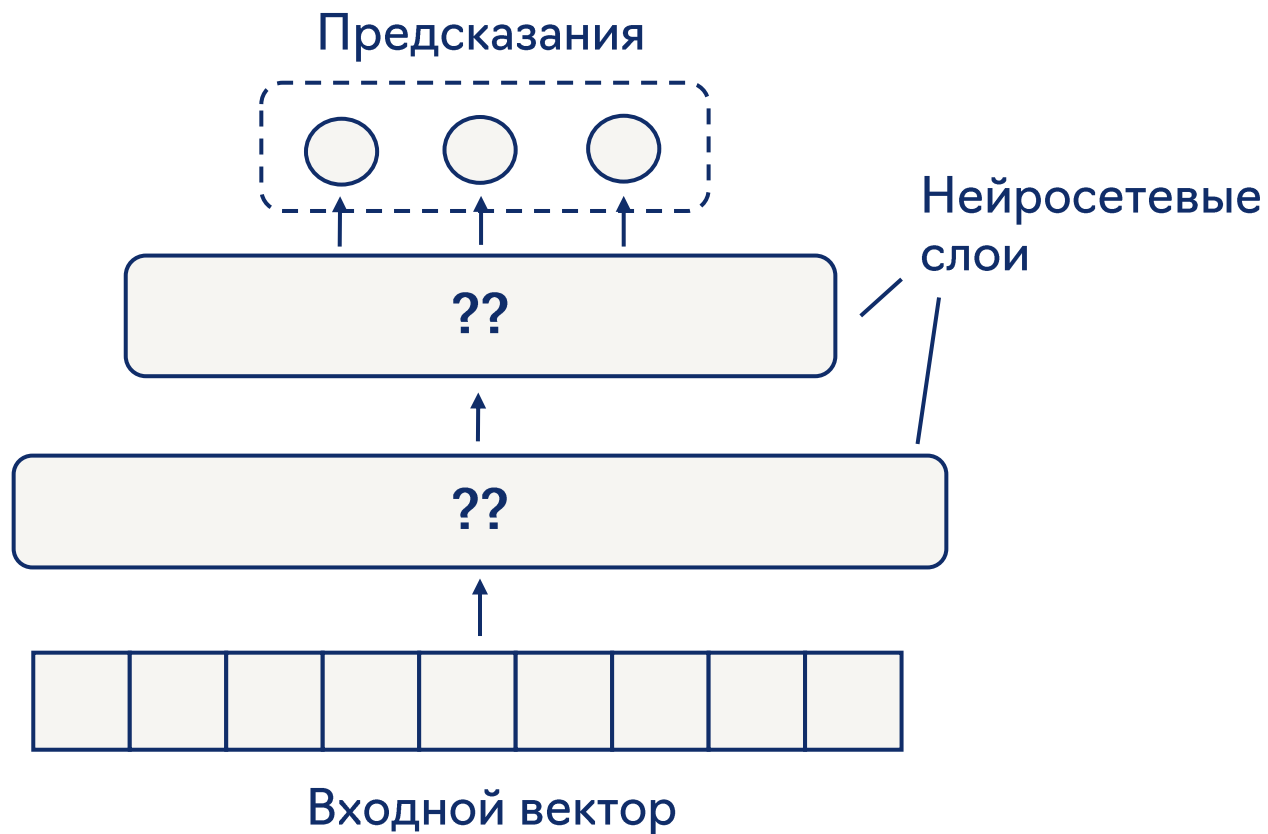
- Нужно учитывать в моделях новый трафик, иначе качество падает
- Если используется градиентный бустинг, то приходится его перестраивать (целиком переобучать)
 - Обычно на одной-двух неделях данных (или на месяце)
- А нейросеть можно дообучать
 - еще и делать это очень быстро

Проблемы нейросетей

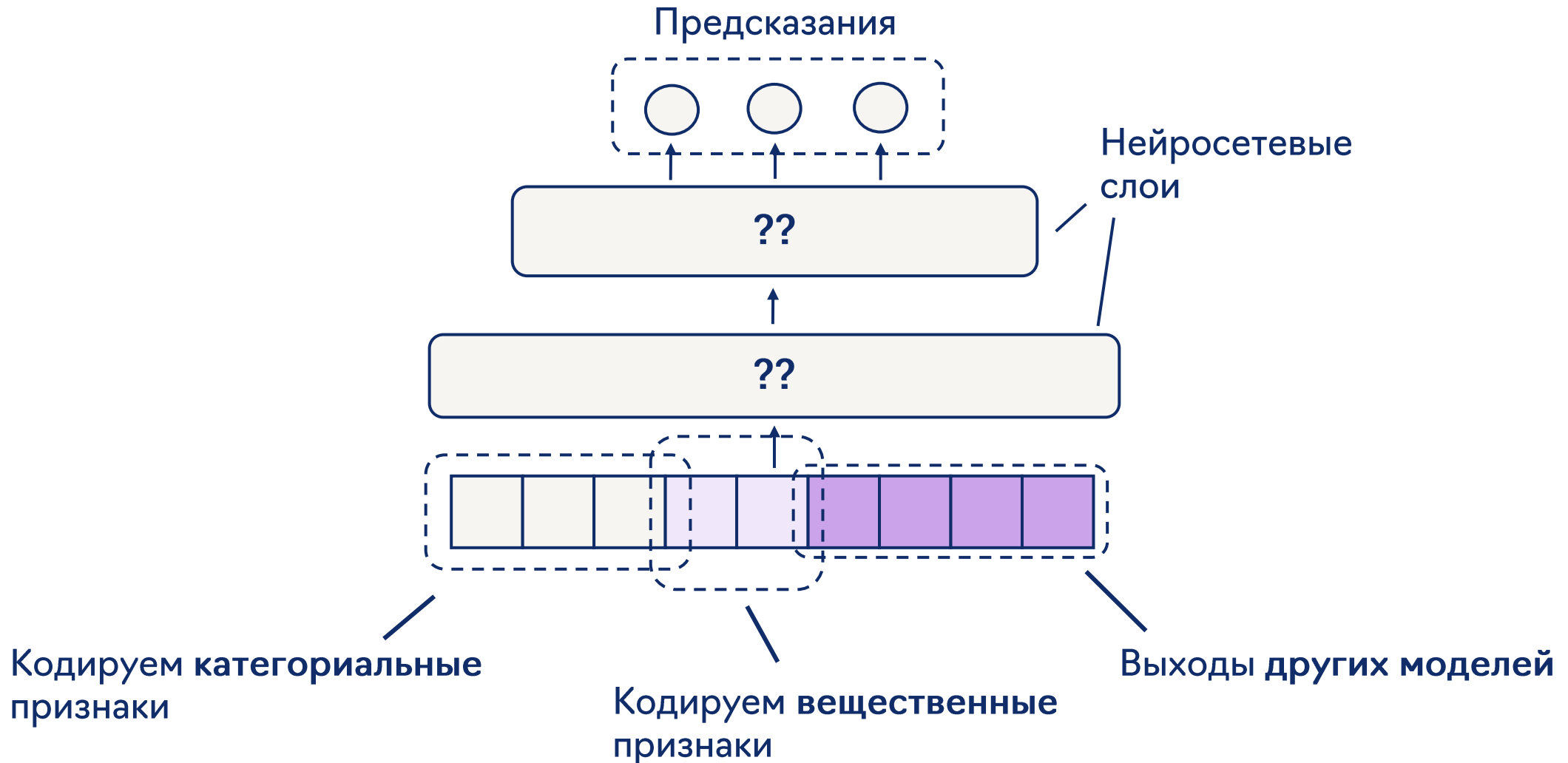
- Высокий порог вхождения
 - Нужно уметь обучать нейросети, работать с GPU, делать распределенные обучения, внедрять нейросети в рантайм
- DL – не серебряная пуля. Когда стоит использовать:
 - Много данных
 - Важны высококардинальные признаки
 - Важны неструктурированные данные
 - Много сигналов
 - Нужно быстро дообучаться

Ранжирующая нейросеть

1. Формируем входной вектор
2. Применяем нейросетевые слои
3. Что-то предсказываем



Вход нейросетевого ранжирования



Категориальные признаки

Часть 2

Категориальные признаки

- Категориальный признак задаётся множеством значений, из которых на каждом сэмпле реализуется какое-то одно
- Кардинальность – количество уникальных значений (мощность множества)
- В рекомендациях кардинальности разнятся от 2 до миллиардов

Категориальные признаки

- Категориальный признак задаётся множеством значений, из которых на каждом сэмпле реализуется какое-то одно
- Кардинальность – количество уникальных значений (мощность множества)
- В рекомендациях кардинальности разнятся от 2 до миллиардов
 - 2 значения – e.g., утро/вечер
 - Миллиард – e.g., item ID

One hot encoding

Представим категориальный признак в виде one hot вектора $e_i = (0, \dots, 1, \dots, 0) \in \{0,1\}^n$.

Применим линейный слой (bias=False):

- То есть умножим вектор на матрицу $W \in R^{n \times d}$

One hot encoding

Представим категориальный признак в виде one hot вектора $e_i = (0, \dots, 1, \dots, 0) \in \{0,1\}^n$.

Применим линейный слой (bias=False):

- То есть умножим вектор на матрицу $W \in R^{n \times d}$
- $e_i W = (W_{ij})_{j=1}^d$ – получаем i -ю строку матрицы W

Получается, что:

- W – матрица обучаемых эмбеддингов, в которой каждому значению признака сопоставляется свой эмбеддинг
- $e_i W$ – операция **embedding lookup**, которая достает нужный вектор из матрицы

Матрица эмбеддингов

0
0
0
1
0

^T
 $\times$

w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}
w_{41}	w_{42}	w_{43}
w_{51}	w_{52}	w_{53}

 $=$

w_{41}
w_{42}
w_{43}

^T

Target encoding: если размерность вектора 1, то можем выучить средний CTR

One hot encoding

Пусть у нас несколько признаков, $e \in \{0,1\}^n, f \in \{0,1\}^m$

- Можно сконкатенировать one hot представления и применить линейный слой
- На выходе будет сумма эмбеддингов
- Минус такого подхода – у всех эмбеддингов одинаковая размерность
- На практике часто делаем **конкатенацию**
 - Если эмбеддинг очень широкий, то сумма может выучить конкатенацию более маленьких эмбеддингов

Матрица эмбеддингов

0	w_{11}	w_{12}	w_{13}
0	w_{21}	w_{22}	w_{23}
0	w_{31}	w_{32}	w_{33}
1	w_{41}	w_{42}	w_{43}
0	w_{51}	w_{52}	w_{53}

×

0	w_{61}	w_{62}	w_{63}
1	w_{71}	w_{72}	w_{73}
0	w_{81}	w_{82}	w_{83}

=

w_{41}
w_{42}
w_{43}

+

w_{71}
w_{72}
w_{73}

The diagram illustrates the one-hot encoding process. It shows a vector e (with values 0, 0, 0, 1, 0) multiplied by a matrix of weights w (with rows $w_{11}..w_{13}$, $w_{21}..w_{23}$, $w_{31}..w_{33}$, $w_{41}..w_{43}$, $w_{51}..w_{53}$, $w_{61}..w_{63}$, $w_{71}..w_{73}$, $w_{81}..w_{83}$). The result is a sum of two vectors: one from the first matrix (rows 4, 5, 6) and one from the second matrix (rows 7, 8, 9). The vectors are shown as columns of boxes with their respective weights.

Размеры эмбедингов

- Можно дать всем признакам одинаковый размер эмбедингов, но это неоптимально
- Размер эмбедингов должен зависеть от:
 - Кардинальности признака – чтобы описать небольшой признак, не нужен широкий эмбединг
 - Информативности признака – его полезности для целевой задачи
- Пусть есть эмбединг из d элементов, каждый из которых описывается с помощью s битов.
 - Суммарно можем описать 2^{ds} разных значений
 - Если кардинальность признака n , то $ds = \log n$

Эвристика от Google: $d = 6 \sqrt[4]{n}$

Dimension optimization:
подбираем размер эмбедингов
в процессе обучения.

Память

- Матрицы эмбедингов много весят
 - В YoutubeDNN – примерно 1 млрд параметров
 - У Цукерберга – до 13 триллионов параметров
- Один признак кардинальности 10 млн с эмбедингом размера 256 – это 10 GB
 - А признаков еще и несколько
 - И кардинальности могут быть больше
- На GPU – не помещается

Шардирование матриц эмбедингов

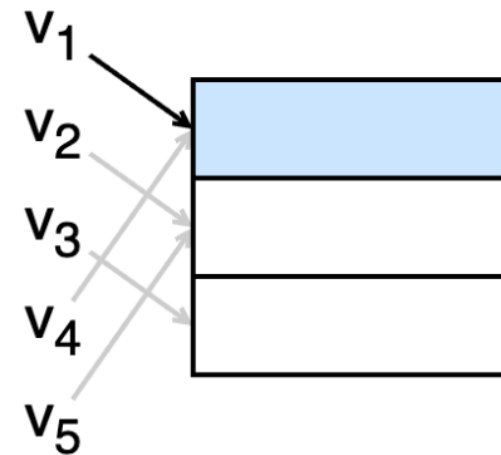
- При распределенном обучении можно делать шардирование
- По координатам (column-wise): хранить на каждой GPU определенное подмножество координат
- По значениям (table-wise): хранить на каждой GPU эмбединги для определенного подмножества значений признака
- Во время forward / backward собираем на каждой карточке нужные эмбединги и градиенты

CPU offloading

- Можно хранить эмбединги в оперативной памяти
 - На машинках для обучения её много – до терабайтов
- При обучении на батче переносим нужные эмбединги на GPU, градиент посылаем обратно на CPU
- Чтобы GPU не простаивало, асинхронно обновляем эмбединги на CPU (не дожидаясь обновления)

Hashing trick

- Пусть можем себе позволить только небольшую матрицу эмбеддингов
- Сделаем случайный маппинг из значений признаков в строки матрицы
 - Как: хэшируем значение признака и берем остаток по модулю M
- Появились коллизии – разные значения признаков попадают в один и тот же эмбеддинг
 - Это дополнительная регуляризация: уменьшаем переобучение, боремся с one epoch phenomenon

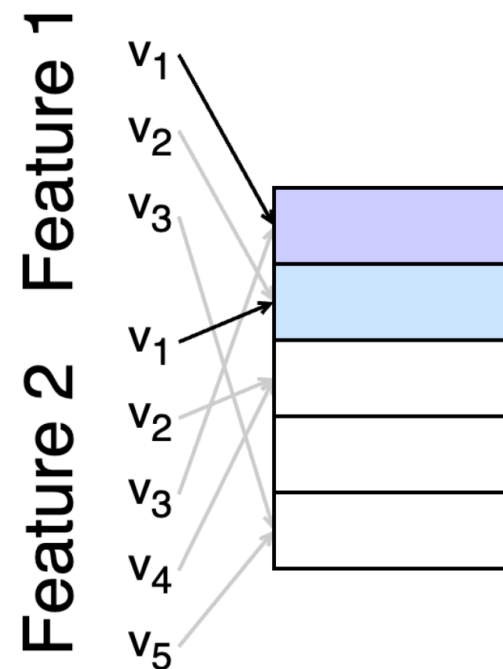


Мультихэш

- Мультихэш – считаем несколько хэшей с разными сидами
 - для каждого признака получаем несколько эмбеддингов
 - агрегируем в единый эмбеддинг с помощью суммы или конкатенации
- Уменьшаем коллизии: вероятность совпадения комбинаций айдишников у двух значений признака – небольшая

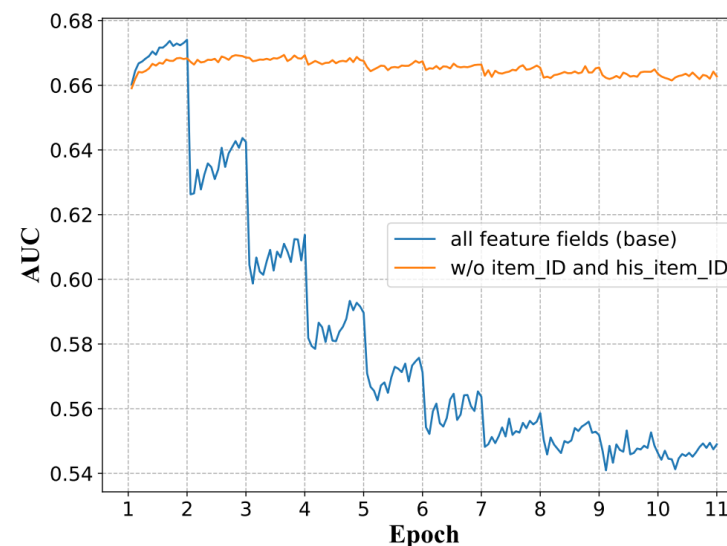
Case study: unified embeddings

- Наша любимая научная группа из Google
- Сделали единую матрицу эмбеддингов для всех признаков
 - У каждого признака свои хэши, но они ведут в одну матрицу
- Чтобы сделать отличающиеся размеры эмбеддингов у признаков, используем разное количество лукапов для разных признаков



Переобучение категориальных эмбеддингов

- Вся та же логика про переобучение и меморизацию, что и для обучаемых эмбеддингов объектов
- Как бороться с переобучением:
 - Регуляризация: dropout, l_2/l_1 —регуляризация
 - Hashing trick тоже помогает
 - Клиппинг: заменяем редкие значения на OOV (out-of-vocabulary) токен
 - Re-initialization: сбрасываем эмбеддинги после каждой эпохи
 - Или заморозить эмбеддинги после первой эпохи



[Towards Understanding the Overfitting Phenomenon of Deep Click-Through Rate Prediction Models](#)

Batched lookup

- Лучше запускать одну большую операцию на GPU, чем много маленьких
 - GPU kernel launch overhead
- Fusion: объединение операций в одну
- Делать отдельный embedding lookup для каждого признака – это дорого
- Можно объединить эмбединги признаков в одну матрицу, и делать единый lookup:
 - Применив сдвиг, при котором каждый признак попадает в свою область матрицы
 - При unified подходе сдвиг не нужен

Sparse gradients

- При обучении, на каждом батче встречается небольшая часть всех эмбеддингов
 - У остальных эмбеддингов получается нулевой градиент
- Sparse optimizer – обновляем параметры только для эмбеддингов с ненулевым градиентом
- Ускоряем коммуникации при распределенном обучении
 - Не делаем пересылки параметров с нулевым градиентом (и самих нулевых градиентов)
- Увеличивайте learning rate для параметров со sparse градиентом

Rowwise optimizer

- В оптимизаторах типа Adam храним на каждый параметр статистики градиента (первый и второй момент)
 - Это $\times 3$ по памяти (а для mixed precision и распределенного обучения – может и $\times 4$)
- Rowwise оптимизатор: храним две статистики на каждый эмбединг

Вещественные признаки

Часть 3

Вещественные признаки

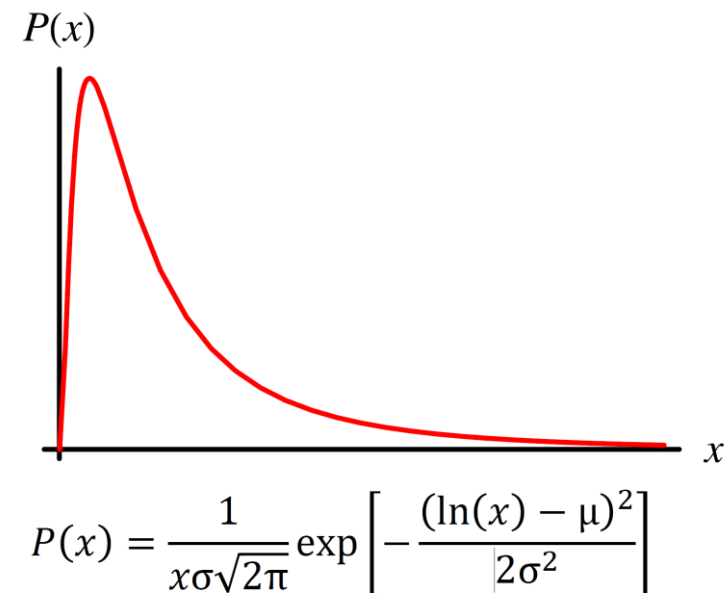
- Большая часть признаков в рекомендациях – вещественные:
 - Счётчики (сколько раз X произошло в ситуации Y)
 - Статистики по множествам (mean/median/std/max/min)
 - Временные статистики (e.g., средняя частота, время с последнего X)
 - Вещественные характеристики пользователей и айтемов (e.g., age, price)
 - Предсказания других моделей (e.g., предсказание двухбашенной модели)

Зачем нужна обработка вещественных признаков?

- Нейросети чувствительны к масштабу признаков
 - У признаков очень разный масштаб
 - Признаки с большими значениями дают большой вклад в градиент и доминируют при обучении
 - Нужна нормализация
- В реальных данных всегда есть выбросы
 - Выбросы ломают обучение нейросетей (либо сильно ухудшают качество)
- Нейросети не умеют работать с пропусками без дополнительной обработки

Log1p

- $F(x) = \log(x + 1)$
- Уменьшаем скошенность распределения
- В реальных данных встречаются логнормальные распределения (e.g., цена)
 - Большая часть значений, принимаемых признаком – небольшие
 - Дифф между 1 и 2 важен, между 100001 и 100002 – уже не очень

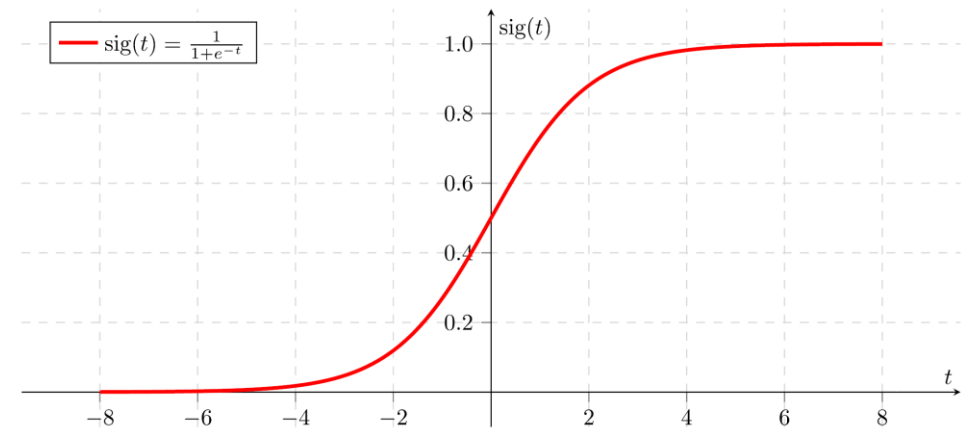


x	1	2	3	10	100	10000
$\log(x + 1)$	0.69	1.1	1.39	2.39	4.61	9.21

Sigmoid squashing

$$F(x) = \sigma(\gamma x + \beta) = \frac{1}{1 + e^{-\gamma x + \beta}}$$

- Squashing – “зжимаем” признак в определенном диапазоне значений (включая выбросы)
- Важно нормализовывать значения перед сигмодой, чтобы не было затухающих градиентов
 - Производная сигмоды близка к нулю на краях
- Можно делать сразу несколько сигмод с разными обучаемыми параметрами γ_i, β_i
 - Получаем эмбединг



Функция распределения

Сопоставим каждому значению признака его значение функции распределения:

- $F(x) = P(X \leq x)$

Функция распределения

Сопоставим каждому значению признака его значение функции распределения:

- $F(x) = P(X \leq x)$
- Приводим признак в равномерное распределение от 0 до 1
 - Убрали скошенность распределения
 - И выбросы
- Как посчитать функцию распределения:
 - Не хотим считать эмпирическую функцию распределения
 - Будем использовать квантильную аппроксимацию

Периодические функции

$$F(x) = [\sin(2\pi c_1 x), \cos(2\pi c_1 x), \sin(2\pi c_2 x), \cos(2\pi c_2 x), \dots]$$

- Используем для выражения периодических зависимостей
 - E.g., для признаков, связанных с временем
- $c_1, c_2, \dots, c_n$ могут быть фиксированными (обозначать чистоту типа час, день, неделя) или обучаемыми
- Похожий механизм использовался для позиционных эмбеддингов в оригинальном трансформере

Квантизация

Превращаем вещественный признак в категориальный:

$$F(x) = \sum_{i=1}^n 1[a_i < x \leq b_i] i$$

- Разбиваем значения признака на интервалы (бины) по квантилям
- Кодировем значение признака номером бина, в который он попал
- Это кусочно постоянное кодирование

Теряем информацию об отношении порядка – как между бинами, так и внутри бина.

Кусочно-линейное кодирование

$$\text{PLE}(x) = [e_1, \dots, e_T] \in \mathbb{R}^T$$
$$e_t = \begin{cases} 0, & x < b_{t-1} \text{ AND } t > 1 \\ 1, & x \geq b_t \text{ AND } t < T \\ \frac{x - b_{t-1}}{b_t - b_{t-1}}, & \text{otherwise} \end{cases}$$

- Кодируем не только номер бина, но и сохраняем порядок между бинами и внутри бина
- Если у нас есть матрица, на которую мы умножаем этот вектор, то как перейти от умножения к lookup?

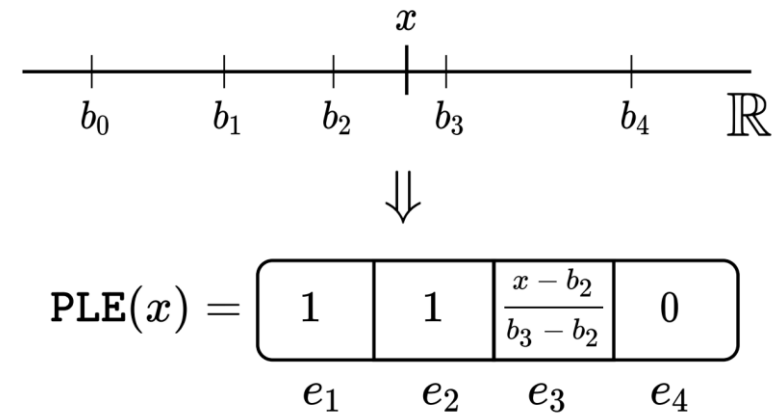


Figure 1: The piecewise linear encoding (PLE) in action for $T = 4$ (see Equation 1).

Гладкость распределений

4.2 Feature distribution

In addition to mapping features to a restricted numerical range, we ensured most of them had a smooth distribution. Why obsess over the smoothness of distributions? Below are some of our reasons.

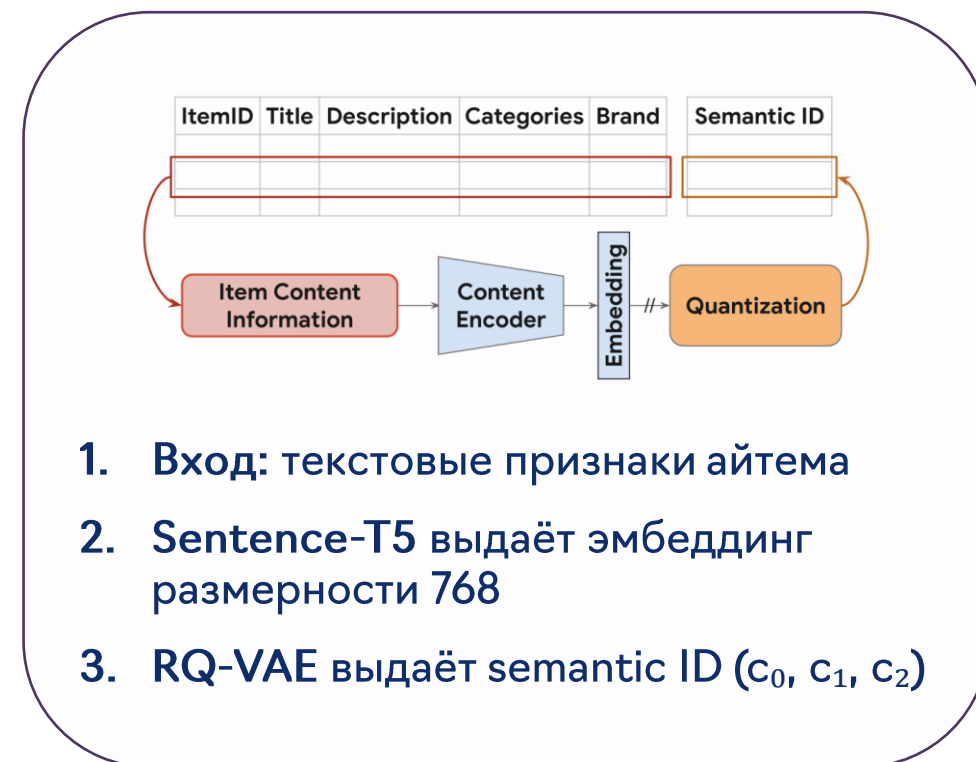
4.2.1 Spotting bugs. When dealing with hundreds of millions of feature samples, how can we verify a small fraction of them are not buggy? Range checks are useful but limited. We found smoothness of distribution an invaluable tool to spot bugs as the distribution of errors often stand in contrast to the typical distribution. To give an example, we had bugs related to currencies in the prices logged for certain geographies. And for periods greater than 28 days, the logged price was the monthly price instead of the daily price. These errors showed up as spikes on the initial distribution plots.

Предобученные эмбединги

- L2-нормализация
- Для понижения размерности эмбединга используем PCA
- Векторная квантизация: превращаем эмбединг в категориальные признаки
 - Сопоставляем векторам элементы кодабука
 - E.g., обучаем kmeans и используем номер центраида как признак
 - Semantic IDs

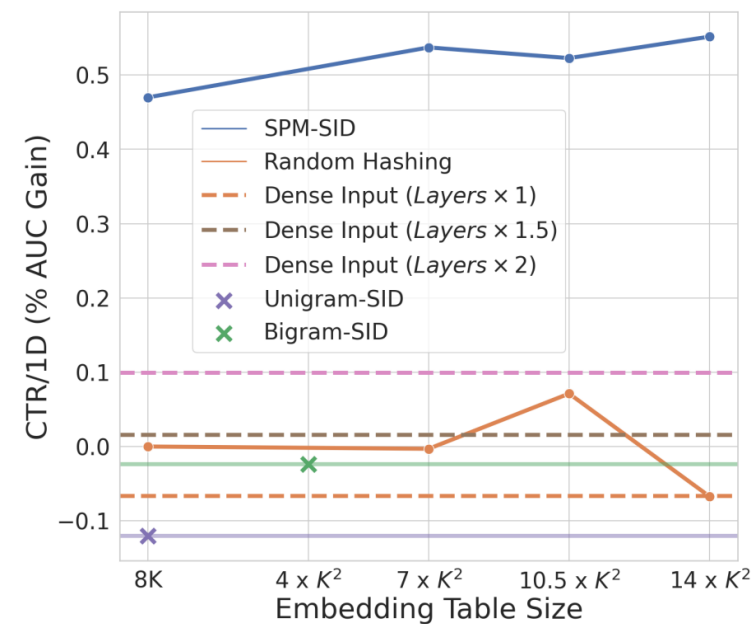
Semantic IDs

- Проблема хэширования – получаемые последовательности айдишников не несут семантики
 - Для новых айтемов мультихэш выдает какую-то неизвестную для модели комбинацию
 - Сильный упор в меморизацию
- А что если бы у нас была интерпретируемая последовательность айдишников?
 - Иерархия: на первом уровне – общая категория, затем сужаем категорию, etc
 - Это inductive bias!
 - Ровно такой метод и предложили ученые из Google; под капотом – дискретный вариационный автокодировщик



Case study: YouTube ranking

- Главный признак в ранжирующей модели Ютуба – item ID
 - Раньше использовали hashing trick
- Заменяли его на semantic IDs:
 - Обучили RQ-VAE поверх контентных эмбеддингов видео (VideoBERT)
- Результат: значительный прирост качества на холодном срезе
- Прямое использование контентного эмбеддинга работает хуже



(b) Cold-start CTR/ID AUC

Выводы

Обсудили:

- Почему для ранжирования нужны нейросети
- Обработку категориальных признаков
- Обработку вещественных признаков
- Выходы других моделей

На семинаре разбор статей:

- Unified Embeddings
- Piecewise Linear Encoding